DD: WO-01 — Work Order Module (parent)

Developer implementation guide: `DD_WO-01-Work_Order_Module-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the module contract `DD_WO-01-Work_Order_Module-v1.0.md`.

Verdict: ■ New | **Wave:** 2 | **Group III:** Fulfillment & Operations | **Version:** v1.0 | **Module:** Work Order The Work Order module is the system of record for every commercial + technical operation Wananchi runs through a field WO — installations, support visits, relocations, equipment work, NOC escalations. It owns the WO lifecycle (create → assign → in_progress → finalization_pending → completed/cancelled), the assignment + claim mechanism (TECHNICIAN mode + TEAM mode with internal-tech and external-dispatcher claim paths), the multi-phase mechanism (for Shifting's disconnect→reconnect), the structured-notes + attachments + equipment-bindings + warranty model, and the Kafka event surface that downstream modules consume. This parent DD indexes the constellation: one framework DD + three flow satellite DDs. Specifications live in the satellites; this parent carries module-level glossary, condensed narrative, cross-DD coordination, the three-label §5 structure, and the deployment + migration story.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/07_work_order/DD_WO-01-Work_Order_Module-v1.0.md`
Version	`v1.0`
DD type	`module contract`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `POST /api/work-orders`

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;

- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `wo-check-warranty-linkage`
- `wo-enforce-finalize-checklist`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- `WorkOrderEquipmentBindingsRecorded`
- `WorkOrderEscalationCandidate`
- `WorkOrderFinalized`
- `WorkOrderHomePassDetachment`
- `WorkOrderHomePassTopologyResolved`
- `WorkOrderInstallationCompleted`
- `WorkOrderPhaseFinalized`
- `WorkOrderPhaseTransitioned`
- `WorkOrderRPTLinkageRecorded`
- `WorkOrderShiftingCompleted`
- `WorkOrderSupportCompleted`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group III:** Fulfillment & Operations | **Version:** v1.0 | **Module:** Work Order

The Work Order module is the system of record for every commercial + technical operation Wananchi runs through a field WO — installations, support visits, relocations, equipment work, NOC escalations. It owns the WO lifecycle (create → assign → in_progress → finalization_pending → completed/cancelled), the assignment + claim mechanism (TECHNICIAN mode + TEAM mode with internal-tech and external-dispatcher claim paths), the multi-phase mechanism (for Shifting's disconnect→reconnect), the structured-notes + attachments + equipment-bindings + warranty model, and the Kafka event surface that downstream modules consume.

This parent DD indexes the constellation: one framework DD + three flow satellite DDs. Specifications live in the satellites; this parent carries module-level glossary, condensed narrative, cross-DD coordination, the three-label §5 structure, and the deployment + migration story.

1. What this module is

The WO module turns every customer-affecting field operation — new installation, support call, relocation, equipment swap, quality control audit — into a tracked WO row with a defined lifecycle. The module captures who is doing what, where, when, with what equipment, with what outcome. It emits events at every state transition so downstream modules (Customer Service, HomePass, Finance, future Inventory, future SLA engine, future Ticketing) can react without polling.

Position in the SOPHIX V3 architecture. Group III Fulfillment & Operations. Upstream callers: FUL-02 (new-customer onboarding creates installation WOs), future OSR modules (service-add, service-change, disconnect), future Ticketing module (customer-reported issues create support WOs), CSR BO UI (manual WO creation), contractor BO UIs (manual QCS escalation creation). Downstream consumers: ILM-CFG-01 (Customer Service — consumes account-status side-effects), RLM-CFG-01 (HomePass — consumes detachment + topology-resolution events), Finance (contractor invoicing per WO completion), future Inventory (equipment-binding events when the module ships), future SLA engine.

Design philosophy. One module, one schema, one Kafka event family. The module's surface area is uniform across operator countries (KE, UG, TZ, SN) — operator variation is configuration (BPMN composition, Drools KJARs, catalog rows) rather than code forks. New operators are onboarded by adding rows + deploying their BPMN + KJAR, never by changing module code.

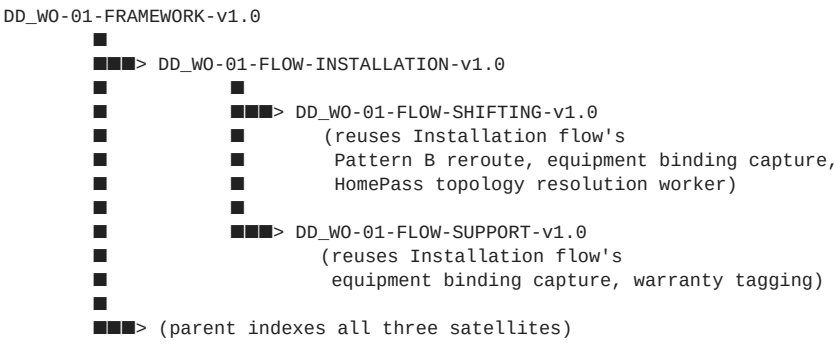
2. Constellation

This module is documented across one framework DD and three flow satellite DDs:

Document	Scope	Lines	Pages
`DD_WO-01-FRAMEWORK-v1.0`	State machine, REST API surface, schemas, contractor + team + technician + schedule + skills model, Kafka event catalog, multi-phase support, generic workers, finalization checklist mechanism, equipment-binding stub	1,651	23
`DD_WO-01-FLOW-INSTALLATION-v1.0`	Installation flow: HFC + GPON + VOIP + Wi-Fi extender + equipment upgrade installs; Pattern B reroute (no-close reassignment); equipment binding capture; HomePass topology confirmation/capture; 6-month warranty tagging	1,147	11
`DD_WO-01-FLOW-SHIFTING-v1.0`	Shifting (relocation) flow: single-WO two-phase mechanism (DISCONNECT → RECONNECT); account-status side-effects (INA at disconnect, STF at reconnect); HomePass detachment + topology update at new address; free-of-charge invariant	878	9
`DD_WO-01-FLOW-SUPPORT-v1.0`	Support flow: customer-reported issues; no-site-visit branch for desk-resolved codes; Pattern A escalation (finalize original + spawn QCS); area-outage exit; 3-month warranty + RPT linkage; TRC/TRD contractor-billing codes	996	10

The framework DD is the foundation; each flow satellite consumes the framework's primitives and adds flow-specific BPMN + Drools + operator-config + flow-specific workers + flow-specific events.

2.1 Dependency order



Framework ships first. Installation flow ships second (most-referenced by other satellites and by FUL-02 STEP-INSTALL). Shifting and Support can ship in parallel after Installation is settled.

2.2 Per-operator deployment

Each operator deploys their own BPMN files + Drools KJARs + operator-scoped catalog rows:

Asset	KE (WIK)	UG (WUG)	TZ (WTZ)	SN (YASSN)
Install BPMN	`WorkOrder_Installation_WIK.bpmn` ■ v1.0	future	future	future
Install KJAR	`wo-installation-wik-v1.0.kjar` ■ v1.0	future	future	future
Shifting BPMN	`WorkOrder_Shifting_WIK.bpmn` ■ v1.0	future	future	future
Shifting KJAR	`wo-shifting-wik-v1.0.kjar` ■ v1.0	future	future	future
Support BPMN	`WorkOrder_Support_WIK.bpmn` ■ v1.0	future	future	future
Support KJAR	`wo-support-wik-v1.0.kjar` ■ v1.0	future	future	future

Asset	KE (WIK)	UG (WUG)	TZ (WTZ)	SN (YASSN)
Catalog seeds	per §3.x of each satellite ■ v1.0	future	future	future

YASSN (Senegal) is expected to ship next, given the cross-country topology variation is most pronounced there (last-mile deployed during install rather than pre-built).

3. Module-level glossary

Terms that span the entire module. Flow-specific terms live in each satellite's glossary.

Term	Meaning
WO (Work Order)	A tracked unit of field-or-back-office work tied to one customer + one HomePass + one job type. Each WO has a `wo_id` (ULID), a kind (INSTALLATION / SHIFTING / SUPPORT), a job_type_code (per-kind catalog entry), a status (PENDING / ASSIGNED / IN_PROGRESS / FINALIZATION_PENDING / COMPLETED / CANCELLED), and an originating context.
Kind	Top-level WO category. v1.0 ships three kinds: INSTALLATION, SHIFTING, SUPPORT. Each kind has its own BPMN + Drools KJAR + flow satellite DD. Future kinds (e.g., NETWORK_MIGRATION) can be added without changing the framework.
Job Type	The specific work code within a kind. INSTALLATION has H01 (HFC install), GIN (GPON internet), G07 (VOIP), etc. SHIFTING has GSD/GSR/HSD/HSR. SUPPORT has GP3, HS1, QCS, RPT, etc. Stored in `wo_job_type_catalog` per operator. The Job Type drives required notes/attachments at finalization, the network type (HFC/GPON), and whether a site visit is needed.
Operator	A Wananchi-group operating country. v1.0 ships WIK (Kenya); WUG (Uganda), WTZ (Tanzania), YASSN (Senegal) are future. The framework is operator-agnostic; per-operator behavior lives entirely in configuration + BPMN + Drools.
Contractor	An entity that runs field teams executing WO work. Internal contractors are Wananchi in-house teams (`ctr_Z071H` Karen Rapid Response, `ctr_NOC_MAINT` NOC Maintenance). External contractors are third-party companies (`ctr_GTECH`, `ctr_ADRIAN`). Distinguished by `wo_contractor.is_internal` boolean. Affects whether individual technicians are tracked (yes for internal CDD staff; optional for external freelance crews).
Team	A unit within a contractor. WOs are assigned to a team (and optionally to a specific technician within the team). The team is the practical assignment grain — both internal and external contractors operate via teams.
TECHNICIAN mode / TEAM mode	Two assignment modes. TECHNICIAN: a specific tech is named at assignment time (collapses claim into the assignment). TEAM: only contractor + team named at assignment; first claimer wins via race-to-claim (internal-tech path) or contractor's dispatcher acknowledges receipt (external-dispatcher path).
Pattern A vs Pattern B	Two escalation patterns for stuck work. Pattern A (Support flow): finalize the original WO + create a new escalation WO (QCS) under a different team's ownership. Pattern B (Installation flow): keep the same `wo_id` open, reassign it across teams without closure, return to the original team for final finalization.
Phase / current_phase	A sub-state within `status=IN_PROGRESS`. Used by multi-phase flows (currently only Shifting). Cycles `NULL` → `DISCONNECT` → `RECONNECT` over the life of one Shifting WO. Stored on `work_order.current_phase`; phase transitions emit `WorkOrderPhaseTransitioned` events.
Equipment binding	A row in `wo_equipment_ref_stub` capturing what hardware was physically installed/replaced/recovered/reused during the WO. Brand, model, serial, MAC, ONU-ID, action. Captured at finalization. Consumed by Finance (contractor invoicing per equipment) and future Inventory (stock reconciliation). v1.0 stub; retires when Inventory module ships.
HomePass topology	The physical network path data for serving a customer: OLT port + PBO ID + splitter position + drop ID + distance. In KE, captured at HomePass creation time by network planning; tech confirms or corrects at install. In SN, captured by the field tech during install via "app terrain" mobile app. Either way, the WO module's `WorkOrderHomePassTopologyResolved` event updates the HomePass record at install finalization.
Warranty window	The period after a WO completes during which a subsequent WO at the same customer is considered a "repeat within warranty." 6 months for Installation kind, 3 months for Support kind. Stored as `wo_job_type_catalog.warranty_days`. The Support flow's `wo-check-warranty-linkage` worker uses this to spawn RPT linkage WOs.

Term	Meaning
Finalization checklist	Per-(operator, kind, job_type) configurable set of required notes + attachments that must be present before a WO can complete. Enforced by framework's `wo-enforce-finalize-checklist` worker at second-confirm. The checklist mechanism is the foundation for Wananchi's field-work discipline.
App terrain (SN context)	The Wananchi field-tech mobile application used in Senegal (and emerging in other deployments). From the WO module's perspective, just another REST client of the WO API. Posts notes + attachments + equipment bindings via the same endpoints BO UIs use.

4. Condensed narrative

A typical customer's experience with Wananchi triggers a sequence of WOs over their lifecycle. Reading top-to-bottom:

Day 0 — Customer signs up. FUL-02 STEP-INSTALL creates an `INSTALLATION` WO (kind=INSTALLATION, job_type=H01 for HFC or GIN/GTV for GPON, originating_context=ORDER). The Installation flow runs: Drools picks contractor + team based on the customer's tech region; team's tech claims; tech does the work; captures readings, attachments, equipment bindings, and topology confirmation/capture; first-confirms; second-confirms. The WO closes `COMPLETED` with `warranty_expires_at = today + 180 days`. FUL-02 STEP-ACTIVATE consumes `WorkOrderInstallationCompleted` and triggers provisioning.

If the install can't be brought up (signal issue, missing drop, last-mile gap), the WO doesn't close. Pattern B fires: the same `wo_id` is reassigned to maintenance or construction; they fix the upstream issue; the WO returns to the original install contractor; tech retries; closes successfully. One `wo_id` throughout.

Day 30 — Customer reports slow internet. Future Ticketing module (or v1.0's CSR via BO UI) creates a `SUPPORT` WO (kind=SUPPORT, job_type=GP3 for GPON internet service call). The Support flow's first step checks warranty: customer is within 180 days of their install → an `RPT` linkage WO is spawned alongside (separate `wo_id`, `master_wo_id` pointing to the original install). Both WOs progress in parallel.

The GP3 WO runs through the normal Support flow: contractor + team assignment, tech visit, captures findings + readings + solution. First-confirm with `final_reason=RESOLVED_ON_SITE`. Drools resolution-gate returns `RESOLVED`. Second-confirm. WO closes. `WorkOrderSupportCompleted` event emitted to Finance; Finance applies warranty billing rules (visit may be free or reduced-rate because customer is under install warranty). The RPT WO closes immediately after with `RPT_LINKAGE_RECORDED`.

Day 90 — Customer reports complete service loss. Another `SUPPORT` WO (kind=SUPPORT, job_type=GP3). Warranty check still finds install within 180 days → another RPT WO spawned. This time the contractor finds the issue is upstream of the customer (optical fault at PBO). They cannot resolve. They finalize with `final_reason=COULD_NOT_RESOLVE_ESCALATED`. Drools resolution-gate returns `NOT_RESOLVED_ESCALATE`. `WorkOrderEscalationCandidate` event fires.

Pattern A kicks in: a new `QCS` WO is created (manually by contractor BO UI in v1.0, or automatically by future Ticketing). The QCS WO (kind=SUPPORT, job_type=QCS, `master_wo_id` = original Support WO) routes to NOC Maintenance. NOC verifies optics remotely, finds the line fault, dispatches their own maintenance team. Maintenance fixes the splice. QCS closes `MAINTENANCE_RESOLVED`. Two completed WOs: the contractor's original Support visit (Finance invoices per their contract) + NOC's QCS (Finance tracks separately for internal team productivity).

Day 200 — Customer relocates within the same city. CSR creates a `SHIFTING` WO (kind=SHIFTING, job_type=GSD for GPON disconnect at old address). The Shifting flow runs: single `wo_id`, two phases. DISCONNECT phase finalizes administratively (no site visit) — emits `WorkOrderPhaseFinalized` carrying `accountSubStatusChange={to:INA}`, ILM-CFG-01 inactivates the account; emits `WorkOrderHomePassDetachment`, RLM-CFG-01 clears the customer from the old HomePass. Status remains IN_PROGRESS. `current_phase` transitions to RECONNECT, `job_type_code` updates GSD → GSR. RECONNECT phase runs as a full install-like flow: contractor + tech assigned to NEW HomePass tech region; tech goes to new address; captures readings + new-address note + topology confirmation/capture; first-confirms; client-up gate passes; second-confirms. `WorkOrderPhaseFinalized` carrying `accountSubStatusChange={to:STF}` reactivates the account at the new address. `WorkOrderHomePassTopologyResolved` updates the new HomePass with the as-installed topology. Status → COMPLETED. One `wo_id` across the whole 30-hour move.

This is the rhythm of Wananchi's operations. Every customer-affecting event creates or progresses a WO; every WO emits events at state transitions; downstream modules react. The module is the spinal column of field operations.

5. Module-level coverage

5.1 KE workflow target

The Wananchi KE Work Order module (workshop 2026-04-29 + Bildad Gitonga transcript 2026-04-30) requires:

- Three WO kinds — INSTALLATION, SHIFTING, SUPPORT — each with their own job-type catalog, finalization rules, and flow.
- Contractor + team + technician model with internal (Wananchi in-house) and external (third-party freelance) variants. External contractors are not required to register individual technicians (most use rotating freelance staff that change companies frequently).
- Two assignment modes (TECHNICIAN-mode pre-assigned + TEAM-mode race-to-claim or dispatcher-ack). External-team WOs may use the external-dispatcher claim path.
- Day-of-week + working-hours schedule at team and (optionally) technician scope.
- Structured notes with operator-scoped JSON Schema validation, plus generic free-text notes for context.
- Required-before-finalize checklist enforced by configuration, not by code.
- 2-step finalization (saves on first, closes on second after checklist passes).
- Equipment bindings captured at finalization with brand/model/serial/MAC/ONU-ID/action.
- HomePass topology confirmation (KE) or capture (SN-style) at install + shifting reconnect finalization, with HomePass record updated at WO close.
- Pattern A escalation for Support (two-WO sequence, QCS to NOC).
- Pattern B reroute for Installation (single-WO, reassign without closure).
- Single-WO two-phase mechanism for Shifting (DISCONNECT → RECONNECT in one process instance).
- 6-month installation warranty + 3-month support warranty + RPT linkage on within-warranty repeats.
- TRC/TRD contractor billing codes for warranty visits.
- Configurable per-operator BPMN composition + Drools KJARs + catalog rows so the same module code serves WIK / WUG / WTZ / YASSN.

5.2 TZ codebase coverage today

To be filled in after codebase review. No claims about what the TZ-deployed Sophix codebase does are recorded here until the codebase has been read directly.

The cartography phase (per the pending plan) will produce this content. Once the codebase is available, this section will be filled with verified statements about which WO-module capabilities exist and which do not — replacing this placeholder.

5.3 V3 design rationale

This module's V3 design choices follow from the KE workshop requirements (§5.1) and the cross-country deployment-model variation discussed throughout the constellation:

1. **Operator variation is configuration, not code.** Six variation axes (BPMN composition, Drools KJAR, job_type_catalog, note_kind_registry, finalization_requirements, final_reason_catalog) per the satellites' §3.6. New operators onboard via these axes — never a fork of module code.
2. **Three flow satellites + one framework** keeps each flow's specification independently buildable. Framework owns primitives; satellites own their flow-specific BPMN + Drools + workers + events. Dev teams can work on different satellites in parallel after the framework is settled.
3. **Multi-phase BPMN supports Shifting natively.** The framework's `current_phase` column + `WorkOrderPhaseTransitioned` event + the phase-update internal primitive are designed specifically for Shifting's DISCONNECT→RECONNECT pattern. Future flows that need similar (network migration, multi-step provisioning) can reuse the same primitives.
4. **Pattern A and Pattern B are distinct, intentional escalation patterns.** Pattern A (Support): two WOs, finalize-and-spawn. Pattern B (Installation): one WO, reassign-without-closure. Each pattern matches the operational reality of its kind — Support's work genuinely transfers (contractor → NOC), so two WOs reflects two teams' ownership; Installation's work stays with the install contractor end-to-end, so one WO reflects one team's accountability.
5. **External contractors don't require tech registration.** Internal contractors (Wananchi CDD staff) have BO logins and registered technicians. External contractors typically work with rotating freelancers whom Wananchi doesn't track. The

framework's TEAM-mode external-dispatcher claim path handles this — `assigned_technician_id` stays NULL; the contractor's dispatcher BO user is recorded as the claimer.

6. **Equipment-binding stub is intentional.** v1.0 has no Inventory / Equipment Catalog module. The stub captures enough to satisfy Finance (per-equipment contractor invoicing) and future-Inventory needs (stock reconciliation by serial). When Inventory ships, the stub retires and bindings move to first-class SKU references. Memory rule #16 documents the retirement path.
7. **Cross-country topology variation supported uniformly.** KE pre-plans topology in HomePass and tech confirms; SN captures topology fresh during install via "app terrain" mobile app. Both feed the same `WorkOrderHomePassTopologyResolved` event which RLM-CFG-01 handles in three modes (CAPTURED / CONFIRMED_AS_PLANNED / CONFIRMED_WITH_CORRECTIONS).
8. **All state transitions emit Kafka events.** Downstream modules (Customer Service, HomePass, Finance, future Inventory, future SLA engine, future Ticketing) consume events rather than polling. The module's event surface is the integration contract.
9. **Field Audit feature is NOT used in Wananchi (workshop confirmation).** Service Field Audit is performed indirectly via WO equipment bindings + photos — the existing primitives cover the audit need without a separate feature.

Mapping to TZ baseline (§5.2) — what V3 adds vs what the codebase already supports — will be filled in after codebase review.

6. Cross-DD coordination (module-level)

6.1 Module owner

This module and all its satellites are owned by the Fulfillment & Operations team.

6.2 Upstream callers

Module	What they call	When			
FUL-02 STEP-INSTALL	`POST /api/work-orders` with `kind=INSTALLATION, jobTypeCode=H01\`	GIN\	GTV, originating_context_type=ORDER`	New customer onboarding pays first + install needed	
Future OSR-ADD	`POST /api/work-orders` with `kind=INSTALLATION, jobTypeCode=DLC\`	DL4\	RTV\	EQU, originating_context_type=OSR`	Existing customer adds service, equipment, TV
Future Ticketing module	`POST /api/work-orders` with `kind=SUPPORT, originating_context_type=TICKET`	Customer reports issue; ticket triage decides field action needed			
CSR BO UI	`POST /api/work-orders` with various kinds, `originating_context_type=BACKOFFICE`	Manual WO creation for v1.0 (until Ticketing ships); shifting WO creation always			
Contractor BO UI	`POST /api/work-orders` with `kind=SUPPORT, jobTypeCode=QCS, masterWoId=<original>`	Pattern A escalation (v1.0 manual; future Ticketing-automated)			

6.3 Downstream consumers

Module	What they consume	What they do
ILM-CFG-01 (Customer Service — future)	`WorkOrderPhaseFinalized` (Shifting flow)	Updates `customer_account.sub_status` (INA at disconnect, STF at reconnect)
RLM-CFG-01 (HomePass)	`WorkOrderHomePassTopologyResolved` (Install + Shifting flows), `WorkOrderHomePassDetachment` (Shifting flow)	Updates HomePass record topology + attaches/detaches customer

Module	What they consume	What they do
Finance	`WorkOrderFinalized`, `WorkOrderInstallationCompleted`, `WorkOrderShiftingCompleted`, `WorkOrderSupportCompleted`, `WorkOrderEquipmentBindingsRecorded`, `WorkOrderRPTLinkageRecorded`	Contractor invoicing per visit + per equipment; applies warranty billing rules from TRC/TRD codes; handles RPT-driven warranty-aware billing
Future Inventory / Equipment Catalog	`WorkOrderEquipmentBindingsRecorded`	Stock reconciliation by serial; retires `wo_equipment_ref_stub` (memory rule #16)
Retention	`WorkOrderShiftingCompleted` filtered by `finalReasonCode=NEW_ADDRESS_NOT_SERVICEABLE`	Outreach to customers whose shifting failed at new address
Future Ticketing module	`WorkOrderEscalationCandidate`	Optionally auto-creates QCS WOs (alternative to v1.0 contractor manual creation)
Future FUL-OPS-SLA (Phase 2)	All WO state events	Computes SLA metrics from timestamps
NOC dashboards	All WO events	Operational visibility

6.4 Sibling modules (peer dependencies)

Module	Direction	Contract
RLM-CFG-01 (HomePass)	this module ← RLM	Reads `homepass.tech_region_code` for assignment Drools input; reads existing topology for confirmation in WIK flows
PLM-CFG-01 (Service / Product)	this module ← PLM	Reads `service.product_type` (HFC/GPON) to determine network_type on WO at creation
SIP-01 (Subscriber Information / Package)	this module ← SIP	Reads `customer_subscription` for context at creation (which package the customer has)
SUB-WF-FRAMEWORK-01	this module ↔ framework	Camunda BPMN + Drools KJAR deployment, external task pattern, message correlation

6.5 v1.0 stubs

The framework DD §9 documents three stubs in v1.0:

Stub	What it replaces	Retired by
`staff_notification_stub`	Future ICN-01 Internal Communications module (notifications to BO users about WO state changes)	When ICN-01 module DD is written
`customer_stub`	Future ILM-CFG-01 Customer Service module (customer context lookup, account-status updates)	When ILM-CFG-01 module DD is written
`wo_equipment_ref_stub`	Future Inventory / Equipment Catalog module (first-class equipment SKUs + stock tracking)	When Inventory module DD is written (memory rule #16 documents the retirement path)

7. Deployment + migration

7.1 v1.0 deployment plan

1. **DDL** — create all `wo_*` tables (work_order, wo_notes, wo_attachments, wo_status_history, wo_assignment_history, wo_audit_log, wo_contractor, wo_team, wo_technician, wo_technician_skill, wo_schedule, wo_flow_config, wo_job_type_catalog, wo_note_kind_registry, wo_finalization_requirements, wo_final_reason_catalog, wo_equipment_ref_stub).
2. **Camunda deployment** — deploy three BPMN files: `WorkOrder_Installation_WIK.bpmn`, `WorkOrder_Shifting_WIK.bpmn`, `WorkOrder_Support_WIK.bpmn`.
3. **Drools KJAR deployment** — deploy three KJARs: `wo-installation-wik-v1.0.kjar`, `wo-shifting-wik-v1.0.kjar`, `wo-support-wik-v1.0.kjar`.
4. **Config seed** — populate all catalog rows from §3 of each satellite DD.
5. **Service deployment** — deploy the WO module's REST service + Camunda external task workers.
6. **Cutover** — FUL-02 STEP-INSTALL flips `useStub=false` to call the real `POST /api/work-orders`; CSR BO UI gains "Raise Shifting WO" + "Raise Support WO" workflows; contractor BO UIs surface WOs in their queues with

claim/start/finalize actions.

7.2 Future operator onboarding

Onboarding WUG / WTZ / YASSN requires (per operator):

1. Author operator-suffixed BPMN files (`WorkOrder_Installation_WUG.bpmn`, etc.).
2. Author operator-suffixed Drools KJARs.
3. Add operator-scoped rows to all catalogs.
4. Deploy and cut over.

No module code changes. The framework + flow satellites' design supports the variation surface entirely through configuration.

7.3 Future module retirement of stubs

Per §6.5 and memory rules #16:

- ICN-01 ships → `staff_notification_stub` retired; framework's notification calls switch to real ICN-01 events.
- ILM-CFG-01 ships → `customer_stub` retired; framework's customer lookups switch to ILM-CFG-01; account-status side-effect consumers (currently no-op) become live.
- Inventory module ships → `wo_equipment_ref_stub` retired; `equipment_kind` becomes FK to Inventory's SKU catalog; brand/model collapse into SKU; new module owns stock tracking + transfer + return flows; WO module retains only `(wo_id, equipment_ref)` linkage. Migrate historical stub rows by mapping `(brand, model)` tuples to SKUs. Preserve `WorkOrderFinalized` event payload shape for consumer compatibility.

7.4 Future expansions

- **New WO kinds.** Network migration (HFC → GPON in place), customer disconnect (termination/churn), VIP onboarding, bulk-account operations. Each new kind is a new flow satellite DD + BPMN + Drools KJAR + operator-config rows. Framework primitives unchanged.
- **Ticketing module.** Will own the customer-reported-issue lifecycle; creates Support WOs from tickets. Removes the v1.0 dependency on CSR manual WO creation. Pattern A QCS spawning can move from contractor-manual to Ticketing-automated.
- **Inventory module.** Retires equipment stub; adds first-class SKU + stock + transfer + return flows.
- **FUL-OPS-SLA (Phase 2).** Native SLA computation from WO timestamps.
- **Per-operator catalog overrides for specific tenant segments.** v1.0 ships uniform catalogs per operator; if a single operator needs different rules for a customer segment (e.g., enterprise vs residential), the framework's per-operator scoping can be extended to per-segment without schema changes (segment becomes another dimension in catalog primary keys).

End of DD_WO-01-Work_Order_Module-v1.0.